



Deakin University

LuminaScale: A Scalable, Event-Driven Enterprise Smart Lighting Platform for Mega-Apartment Complexes Project Proposal

Student Name: Thien Phuc Tran

Student ID: s225043528

Date of Submission: 24/7/2026

Document Version 1.0

High-Level Problem / Problem Description

Consumer-grade smart lighting solutions are architected strictly for isolated, single-family residential properties. When an enterprise attempts to centralize lighting management across a mega-apartment complex or residential district containing tens of thousands of units and millions of devices, conventional synchronous architectures quickly collapse under the weight of traffic. The vision of LuminaScale is to build a highly available, event-driven IoT smart lighting platform capable of scaling horizontally to manage over 50,000 individual apartment units and 1,000,000+ total smart endpoints simultaneously. The platform ensures low-latency execution for critical automation paths while providing centralized control for complex managers and analytics for energy grid operators.

Key Technical Challenges & Scalability Concerns

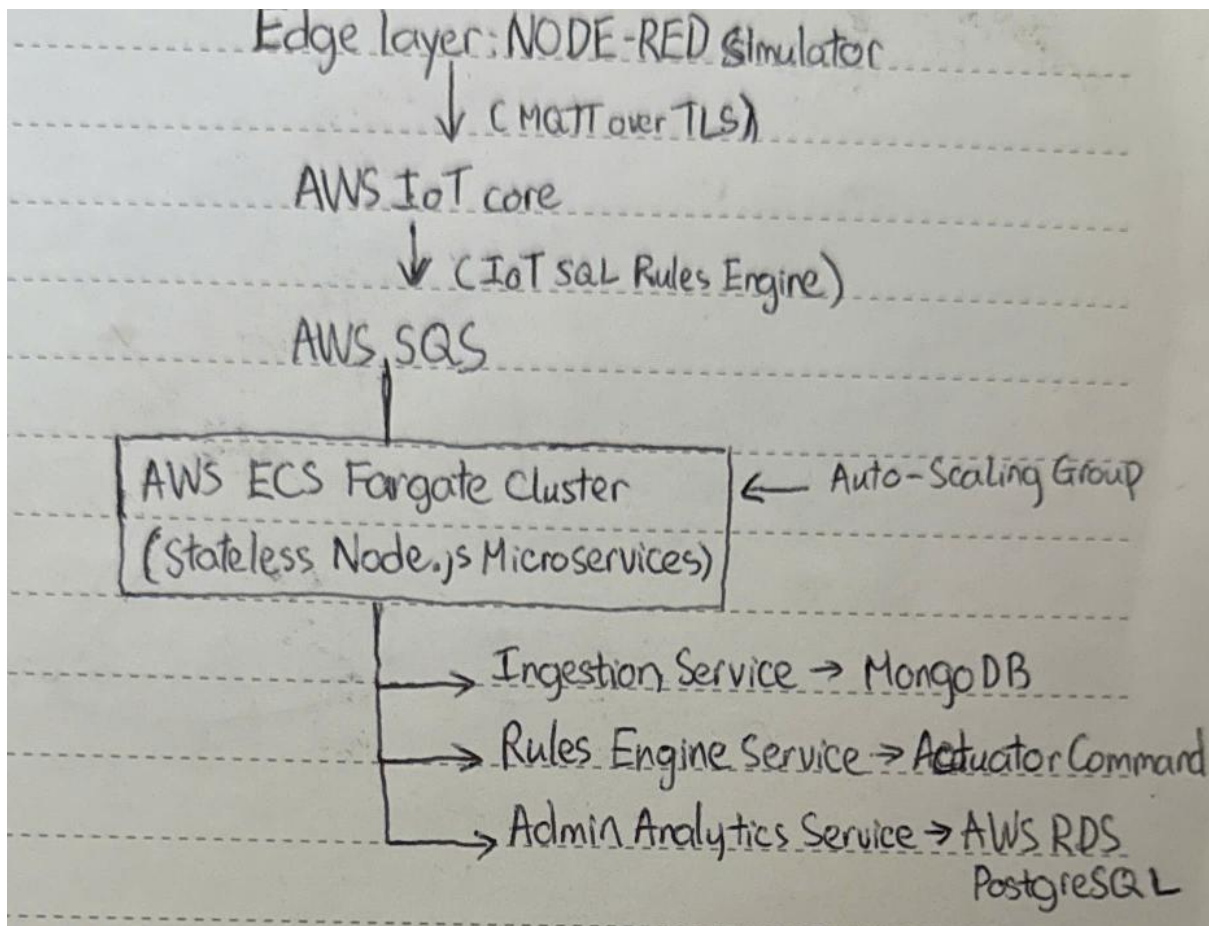
- The "Sunset Spike": As dusk approaches across a city or complex, thousands of ambient light sensors register a drop in lux levels simultaneously. This triggers massive, synchronized MQTT payload blasts and millions of concurrent lighting rule evaluations within a tight 15-minute window.
- Network Storm Recovery: Following a localized power outage or network drop within a multi-story tower, thousands of smart bulbs boot up and attempt to connect, authenticate, and update their states to the cloud at the exact same second, causing extreme ingestion system thrashing.

Differentiators from Existing Solutions

Unlike standard architectures that route every device directly to a single monolith server, LuminaScale utilizes an asynchronous, decoupled event-driven microservice architecture built with Node.js and AWS. Telemetry ingestion is completely separated from rule execution via distributed message queues, ensuring that unexpected spikes in device activity never drop packets or stall system operations.

Solution overview

High-Level Block Diagram Description



The platform is structurally organized into three distinct operational layers:

- **Edge / Simulation Layer (Node-RED):** Simulates arrays of ambient light sensors, PIR motion detectors, and wall dimmers sending payloads via MQTT over TLS.
- **Ingestion & Queue Layer (AWS IoT Core & AWS SQS):** Acts as the high-throughput perimeter gateway. AWS IoT Core ingests MQTT streams and pushes them directly into an AWS SQS queue to buffer traffic.
- **Processing & Compute Layer (Node.js Microservices on AWS ECS):** Stateless microservices containerized via Docker independently handle telemetry writes, automation execution, and structural dashboard reads.

Data Aggregation, Filtering & Processing Algorithms

To prevent system overload and save bandwidth, the solution implements two primary algorithmic stages:

- **Edge-Side State Debouncing:** Motion sensors frequently alternate states within fractions of a second during human movement. A debouncing filter node in Node-RED ensures that state transitions are held for a minimum of 2500ms before generating a cloud-bound message payload.
- **Cloud-Side Moving Average Lux Filter:** To prevent momentary shadows (for example a passing bird or cloud) from falsely activating apartment lights, the incoming lux streams pass through a moving average algorithm before hitting the automation execution node: $Lux_{filtered} = \frac{1}{n} \sum_{i=1}^n Lux_i$

Where n represents the rolling window of historical lux readings. Automation rules only fire if $Lux_{filtered}$ meets the activation threshold.

Scalability Strategy

The architecture guarantees horizontal scaling by making the core Node.js application microservices entirely stateless. By deploying these services into an AWS ECS Fargate Cluster, we configure an AWS CloudWatch Alarm that monitors the depth of the incoming AWS SQS Queue. If the count of un-processed sensor payloads rises rapidly, AWS automatically spins up additional container instances within seconds to share the processing load, scaling back down smoothly when traffic levels return to normal.

Testing Plan

- **Baseline Validation:** Verify correct MQTT data routing from Node-RED to AWS and confirm correct data indexing within MongoDB and PostgreSQL.
- **Load & Scale Stress Testing:** Configure a dedicated Node-RED virtual driver array to generate 20,000 concurrent light/motion event payloads per minute.
- **Success Metrics:** Collect logs showing the AWS CloudWatch Alarm firing, tracking the automatic scaling behavior of the ECS containers as they handle the influx without dropped messages or processing delays.

Implementation Plan

Hardware / Simulation Specifications

- **Edge Platform:** Node-RED runtime environment running on a local development server or an AWS EC2 instance.
- **Simulated Components:**
 - *Sensors:* Ambient Light Level Sub-node, PIR Motion Sub-node.
 - *Actuators:* Smart LED Dimming Controller node (accepts payloads ranging from 0–100% brightness).
- **Communication Layer:** Simulated local Zigbee mesh topology feeding into a simulated building hub. The hub uses MQTT over TLS (Port 8883) to stream structured JSON payloads outward to the designated AWS IoT Core endpoint.

Data Design & Database Schema

NoSQL Tier (MongoDB Atlas): Time-Series Telemetry & States

Selected for its high velocity write handling.

Data collection: ID, Device ID, Building ID, Apartment ID, Timestamp, Power state, Brightness, Current lux, Power consumption.

Relational Tier (AWS RDS PostgreSQL): Relational Infrastructure Configuration

Ensures transactional integrity and strict constraints for core infrastructure layout and user accounts.

Table 1: The "Users" List

This acts like an official building roster. It securely stores who is allowed to log into the system.

- **User ID:** A unique number given to every person, so the system doesn't confuse two people with the same name.
- **Email:** The user's login email address.
- **Password:** A securely scrambled version of their password.
- **Role:** What they are allowed to do.

Table 2: The "Master Device Registry"

This is the master inventory list of every single smart piece of hardware installed in the apartment complex.

- **Device ID:** The unique serial number for a specific smart bulb or motion sensor.

- **Building & Apartment Number:** Exactly where that device is physically located.
- **Device Type:** What the device actually is.
- **Firmware Version:** The software version is currently running on that specific bulb.

Cloud Deployment & Security Blueprint

Ingestion Pipeline: Built using AWS IoT Core configured with an SQL rule action. The data target is mapped straight to an AWS SQS FIFO Queue to preserve message sequence during high traffic load.

Compute: Docker containers running optimized Node.js runtimes deployed on serverless AWS ECS Fargate execution clusters.

Security & Encryption Architecture:

- **Authentication:** Every simulated Node-RED edge hub uses unique, individual X.509 Cryptographic Certificates generated by the AWS IoT Certificate Authority. Unauthorized connections without a valid certificate are rejected immediately at the network edge.
- **Authorization:** Access controls use the AWS IAM Principle of Least Privilege. The Node.js Ingestion Service possesses execution tokens to pull messages from SQS and write records to MongoDB but holds zero system permissions to drop database tables or modify AWS architecture elements.

Project Plan

Week	Focus Area	Core Planned Activities & Deliverables
W1	Requirements & Modeling	Finalize system requirements, map out end-to-end data flows, and outline the core technical architecture specification document
W2	Edge Simulation Setup	Configure the local Node-RED development environment, mock out sensor nodes, and define standard JSON payload parameters.
W3	Cloud Provisioning	Spin up AWS accounts, activate AWS IoT Core, configure core security certificate keys, and verify successful edge-to-cloud MQTT data transfers
W4	Database Engineering	Provision the MongoDB Atlas cluster and setup the AWS RDS PostgreSQL schema; implement secure connection handling within Node.js database drivers.
W5	Microservice Development	Develop the core Node.js ingestion and rules engine microservice applications; bundle the services neatly

		inside Docker container configurations.
W6	Pipeline Integration	Connect the AWS SQS data buffer queues directly to your container clusters, linking ingestion nodes through the system components.
W7	Auto-Scaling Deployment	Configure AWS Auto-Scaling triggers using CloudWatch alarms based on real-time SQS queue metrics.
W8	Stress Testing & Validation	Run load-testing sequences using high-volume Node-RED simulation nodes; capture visual evidence showing system scale and compute growth
W9	Final Report & Handover	Compile experimental data, document code paths within GitHub, and submit the final project report to Ontrack.

Potential Project Risks & Mitigation Actions

Risk 1: Escalating Cloud Infrastructure Expenses

Impact: High. Continuously running high-frequency load simulations on enterprise cloud clusters can run up significant costs.

Mitigation: Configure AWS billing alarms at \$10 intervals. Ensure the ECS cluster is configured with a strict maximum task limit cap, and teardown resources immediately when testing windows complete.

Risk 2: Edge Ingestion Constraints on Simulation Hardware

Impact: Medium. Running thousands of simulated device loop threads simultaneously inside a single local Node-RED process can saturate local CPU limits.

Mitigation: Distribute the workload across multiple distinct virtual Node-RED workflow pages, or utilize an array of lightweight, parallelized command-line scripts to send MQTT messages.

Risk 3: Message Loss During Rapid Auto-Scaling Windows

Impact: High. If container orchestration takes too long to spin up, incoming events could time out or overflow.

Mitigation: Use an AWS SQS queue tier in front of the compute layer. The queue acts as a persistent buffer, storing messages safely for up to a number of days so that no telemetry is dropped while waiting for new server containers to boot up.